

# Problem A: A History of Computing

---

Ten landmark events corresponding to the numbered markers on the computing timeline:

- 1. 1936** : Alan Turing publishes On Computable Numbers : introducing the Turing Machine.
- 2. 1947** : Invention of the Transistor (Bell Labs).
- 3. 1958** : Invention of the Integrated Circuit (IC).
- 4. 1969** : ARPANET is launched : precursor to the modern internet.
- 5. 1981** : IBM Personal Computer (IBM PC) is introduced.
- 6. 1991** : World Wide Web (WWW) becomes publicly available.
- 7. 2000** : Dot-com Bubble crashes.
- 8. 2007** : Apple introduces the iPhone.
- 9. 2012** : Raspberry Pi is released.
- 10. 2022** : Generative AI Revolution : public release of ChatGPT.

## Problem B: Pixel Art

### Solution Description

An  $N \times N$  grid is initialised with zeros. Two pattern modes are implemented:

- **Checkerboard:** assign  $(\text{row} + \text{col}) \% 2$  to each cell : alternating 0s and 1s, top-left always 0.
- **Diamond:** compute centre  $c = N // 2$ . Fill cell  $(i, j)$  with 1 if  $|i-c| + |j-c| \leq c$ , otherwise 0.

**Time Complexity:**  $O(N^2)$  : single pass over all cells.

## Problem C: Moore's Law

---

### (a) Predicted transistor count in 2026

Using the Intel 4004 (1971) as reference:  $T_0 = 2,300$  transistors,  $t = 55$  years elapsed to 2026.

$$T(t) = T_0 \times 2^{t/2}$$

$$T(t) = 2300 \times 2^{55/2}$$

$$2300 \times 2^{27.5} \approx 4.37 \times 10^{11}$$

Predicted transistor count  $\approx$  **437 billion transistors**.

### (b) Comparison with an actual 2026 processor

The Apple M5 (2026) has approximately 45 billion transistors.

Moore's Law predicts  $\approx 437$  billion : roughly **10× larger** than the actual value. The law captures the long-term trend well, but transistor growth has slowed due to physical and manufacturing limits.

### (c) Best-fit doubling period

Solving for the doubling period  $d$  that maps  $T_0 = 2,300$  to 45 billion over 55 years:

$$d \approx 2.5 \text{ years per doubling}$$

The actual growth from the Intel 4004 to a 2026 processor is best described by a doubling period of approximately 2.5 years.

## Problem D: The Simultaneous Strike

---

### (a) What is wrong with the pseudocode?

The pseudocode checks for a KO inside the per-event loop : before all events on the same frame are applied. In the given scenario, both players share frame 512:

(1, 512, 20)    and    (2, 512, 20)

With Player 1 HP = 10 and Player 2 HP = 15, the first event drops Player 1 to 0 and the loop breaks immediately. Player 2's simultaneous attack is never applied, producing an incorrect winner instead of a double KO. The game rule requires all events on the same frame to be fully resolved before any KO is evaluated.

### (b) Corrected Implementation (Python)

The fix groups events by frame, accumulates total damage for both players within each frame, applies all damage simultaneously, clamps HP to 0, then checks for a KO.

**Time Complexity:**  $O(N \log N)$  (one sort + one linear pass).

# Problem E: PageRank in the Age of AI Slop

---

## (a) How PageRank works

### Random-Surfer Interpretation

PageRank models a random web surfer. With probability  $d$  the surfer follows one of the current page's outgoing links; with probability  $1-d$  the surfer teleports to a uniformly random page on the entire web.

### Role of the Damping Factor $d$

$d$  controls the balance between link-following and random teleportation. Without damping ( $d = 1$ ), isolated or dangling pages absorb all rank. Setting  $d < 1$  guarantees every page has a nonzero base probability and ensures convergence.

### Why the Equation is Recursive

The rank of each page depends on the ranks of pages linking to it, which themselves depend on other pages : a circular dependency. No direct closed-form solution exists for general graphs.

$$PR(p_i) = \frac{1-d}{N} + d \sum \frac{PR(p_j)}{L(p_j)}$$

### Computation via Power Iteration

All ranks are initialised to  $1/N$ . The formula is applied repeatedly as a matrix-vector product until values converge (change  $< \epsilon$ ). Convergence is guaranteed for  $d < 1$ .

## (b) Closed-form expression for $S_a$

Let  $\alpha = a/N$  (AI page fraction),  $S_a$  = total PageRank of all AI pages,  $S_h = 1 - S_a$  = total PageRank of human pages.

Human pages link uniformly to all  $N$  pages, so each AI page receives a proportional share of human PageRank. AI pages link only to each other, keeping all AI-internal weight in the AI pool. Writing the aggregate balance equation:

$$S_A = \alpha(1-d) + d(\alpha S_H + S_A)$$

Substituting

$$S_H = 1 - S_A$$

and solving:

$$S_A = \frac{\alpha(1 - d + d\alpha)}{1 - d + d\alpha}$$

The numerator and denominator cancel completely, giving:

$$S_A = \alpha$$

**Result:** The total PageRank of all AI pages equals exactly  $\alpha$  : their fractional share of the web. The damping factor  $d$  cancels out completely.

### (c) Tipping fraction $\alpha^*$

The tipping point is where AI and human pages hold equal total PageRank:

$$S_A = S_H = \frac{1}{2}$$

Since  $S_a = \alpha$  (from part b), the tipping fraction is:

$$\alpha^* = 0.5$$

**For  $d = 0.85$ :** the result is unchanged.  $\alpha^* = 0.5$  regardless of the damping factor.

### Structural Vulnerability

This model shows PageRank is purely link-structure-based and content-agnostic. A large network of AI pages that cross-link heavily accumulates PageRank directly proportional to their share of the web. Once AI pages exceed 50% of all indexed pages they collectively outrank all human content. This is why modern search engines augment PageRank with content-quality, freshness, and user-engagement signals.